QAP, PCP, PoE: Demystifying zk-SNARK Tools

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Recap

2 Quadratic Arithmetic Program

3 Probabilistically Checkable Proofs

4 QAP as a Linear PCP

5 Proof Of Exponent

Recap

Recap: what is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

Recap: what is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

✓ **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.

Recap: what is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- ✓ **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.
- ✓ **Succinctness** — the proof size and verification time is relatively small to the computation size and typically does not depend on the size of the data or statement.

Recap: what is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- ✓ **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.
- ✓ **Succinctness** — the proof size and verification time is relatively small to the computation size and typically does not depend on the size of the data or statement.
- ✓ **Non-interactiveness** — to produce the proof, the prover does not need any interaction with the verifier.

Recap: what is zk-SNARK?

Definition

zk-SNARK

Zero-**K**nowledge **S**uccinct **N**on-interactive **AR**gument of **K**nowledge.

- ✓ **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.
- ✓ **Succinctness** — the proof size and verification time is relatively small to the computation size and typically does not depend on the size of the data or statement.
- ✓ **Non-interactiveness** — to produce the proof, the prover does not need any interaction with the verifier.
- ✓ **Zero-Knowledge** — the verifier learns nothing about the data used to produce the proof, despite knowing that this data resolves the given problem and that the prover possesses it.

Recap: Arbitrary Program To Circuits

We can do that in a way like the computer does it — **boolean circuits**.

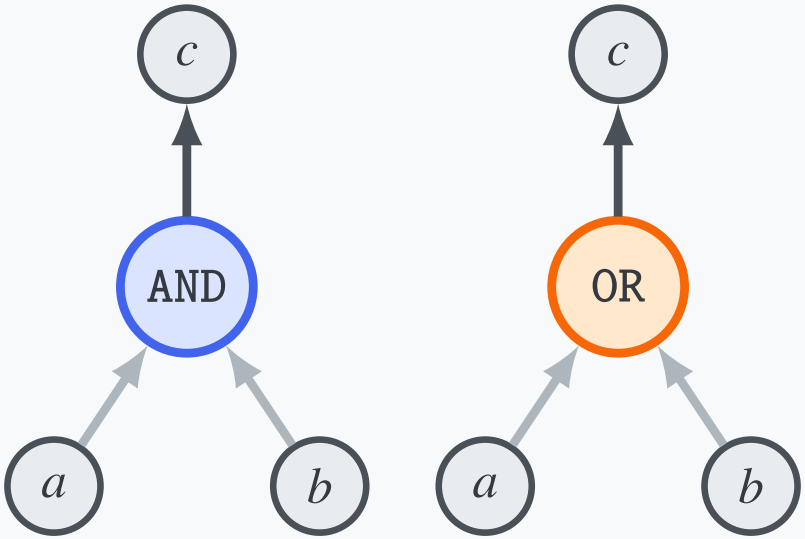


Figure: Boolean `AND` and `OR` Gates

Recap: Arbitrary Program To Circuits

We can do that in a way like the computer does it — **boolean circuits**.

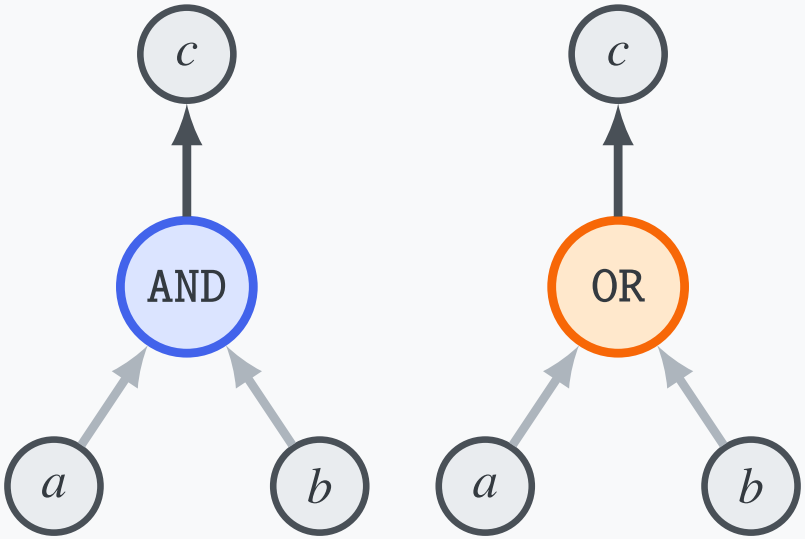


Figure: Boolean `AND` and `OR` Gates

> **100000 gates** just for `SHA256`...

Recap: Arbitrary Program To Circuits

We can do that in a way like the computer does it — **boolean circuits**.

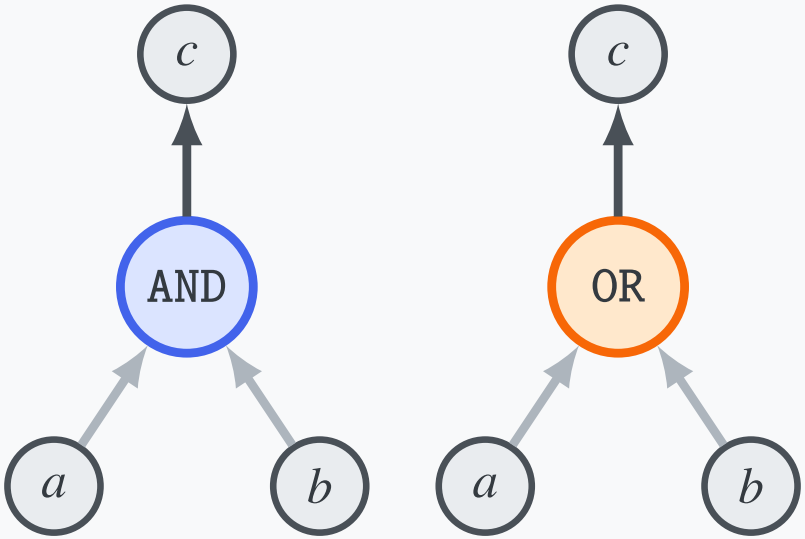


Figure: Boolean `AND` and `OR` Gates

> **100000 gates** just for `SHA256`...

But nothing stops us from using something more powerful instead of boolean values, gates.

Recap: Arbitrary Program To Circuits

Similar to Boolean Circuits, the **Arithmetic Circuits** consist of gates and wires.

- **Wires**: elements of some finite field $\mathbb{F}$.
- **Gates**: field addition ($+$) and multiplication ($\times$).

Recap: Arbitrary Program To Circuits

Similar to Boolean Circuits, the **Arithmetic Circuits** consist of gates and wires.

- **Wires**: elements of some finite field $\mathbb{F}$.
- **Gates**: field addition ($+$) and multiplication ($\times$).

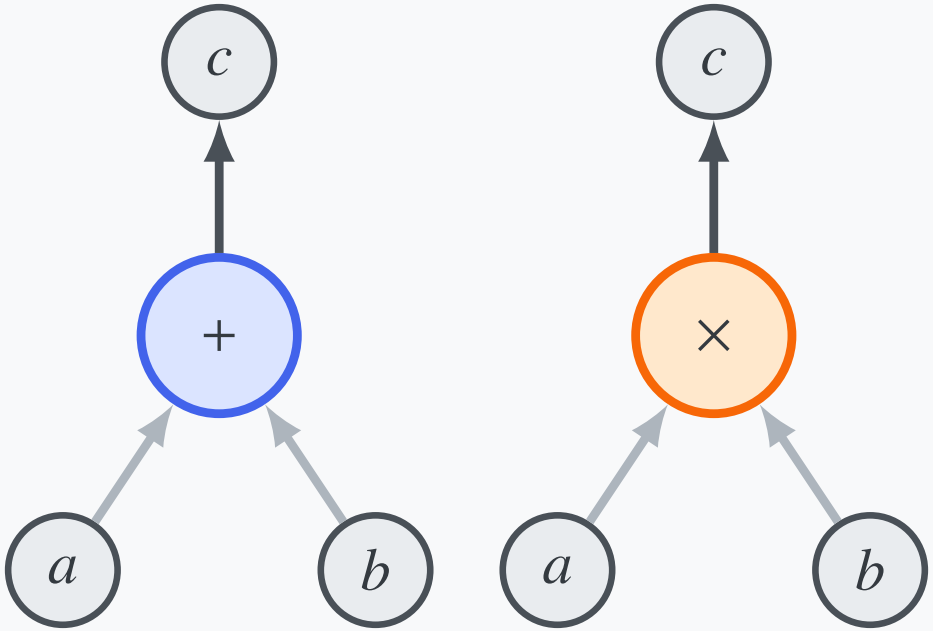


Figure: Addition and Multiplication Gates

Recap: Arbitrary Program To Circuits

Example

How can we translate `if` statements?

```
def example(a: bool, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

Recap: Arbitrary Program To Circuits

Example

How can we translate `if` statements?

```
def example(a: bool, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

We can transform such a function into the next expression:

$$r = a \times (b \times c) + (1 - a) \times (b + c)$$

Recap: Arbitrary Program To Circuits

Example

How can we translate `if` statements?

```
def example(a: bool, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

We can transform such a function into the next expression:

$$r = a \times (b \times c) + (1 - a) \times (b + c)$$

Corresponding equations for the circuit are:

$$r_1 = b \times c, \qquad r_3 = 1 - a, \qquad r_5 = r_3 \times r_2$$

$$r_2 = b + c, \qquad r_4 = a \times r_1, \qquad r = r_4 + r_5$$

Recap: Arbitrary Program To Circuits

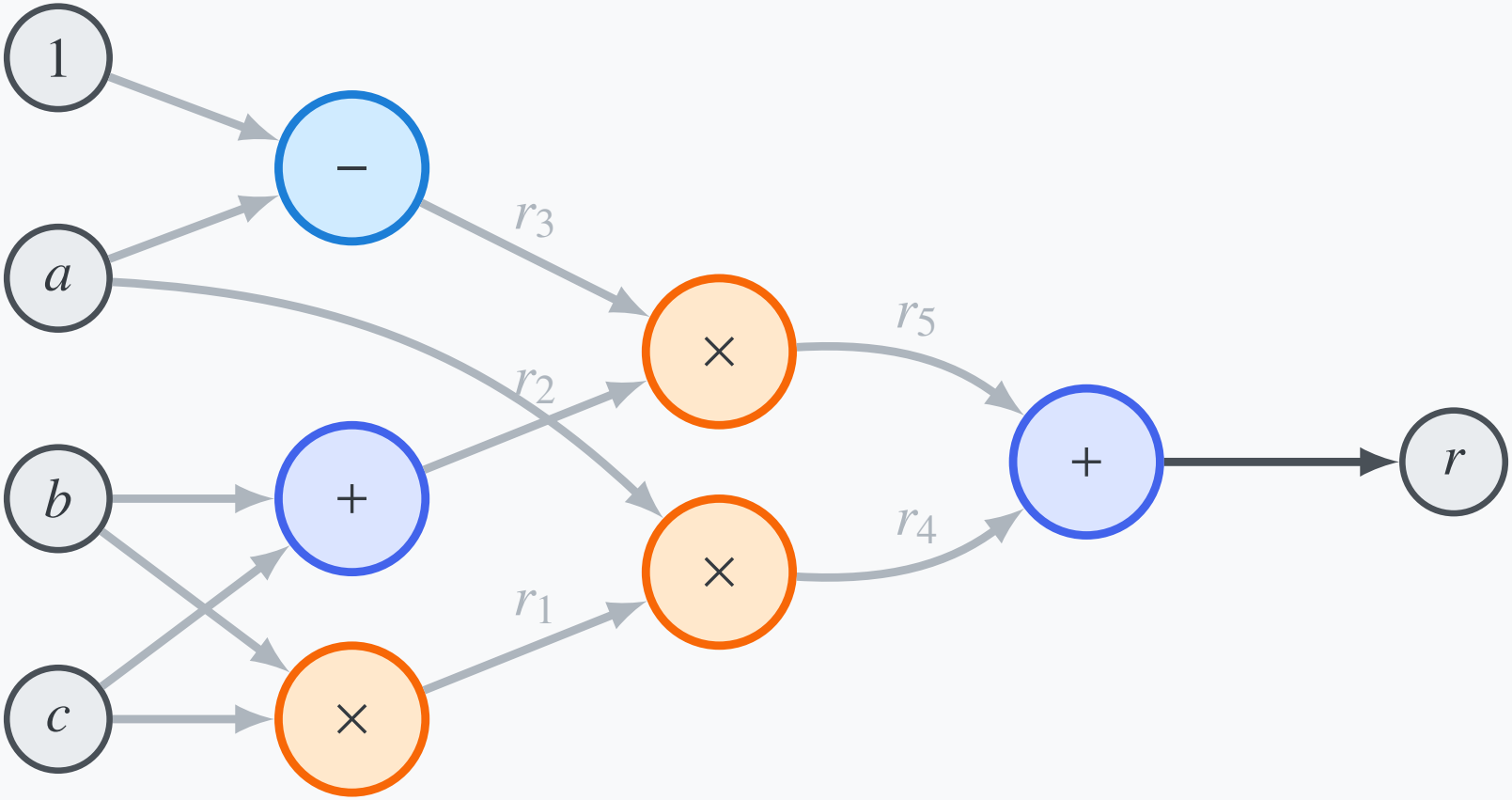


Figure: Example of a circuit evaluating the `if` statement logic.

Recap: R1CS

Each **constraint** in the Rank-1 Constraint System must be in the form:

$$\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$$

Recap: R1CS

Each **constraint** in the Rank-1 Constraint System must be in the form:

$$\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$$

Where $\langle \mathbf{u}, \mathbf{v} \rangle$ is a dot product.

$$\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$$

Recap: R1CS

Each **constraint** in the Rank-1 Constraint System must be in the form:

$$\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$$

Where $\langle \mathbf{u}, \mathbf{v} \rangle$ is a dot product.

$$\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$$

Thus

$$\left(\sum_{i=1}^{n} \ell_i z_i \right) \times \left(\sum_{j=1}^{n} r_j z_j \right) = \sum_{k=1}^{n} o_k z_k$$

That is, actually, a quadratic equation with multiple variables.

Recap: R1CS

Example

Consider the most basic circuit with one multiplication gate: $x_1 \times x_2 = r$. The witness vector $\mathbf{z} = (r, x_1, x_2)$. So

$$z_2 \times z_3 = z_1$$

$$(0 + z_2 + 0) \times (0 + 0 + z_3) = z_1 + 0 + 0$$

$$(0z_1 + 1z_2 + 0z_3) \times (0z_1 + 0z_2 + 1z_3) = 1z_1 + 0z_2 + 0z_3$$

Recap: R1CS

Example

Consider the most basic circuit with one multiplication gate: $x_1 \times x_2 = r$. The witness vector $\mathbf{z} = (r, x_1, x_2)$. So

$$z_2 \times z_3 = z_1$$
$$(0 + z_2 + 0) \times (0 + 0 + z_3) = z_1 + 0 + 0$$
$$(0z_1 + 1z_2 + 0z_3) \times (0z_1 + 0z_2 + 1z_3) = 1z_1 + 0z_2 + 0z_3$$

Therefore the coefficients vectors are:

$$\boldsymbol{\ell} = (0, 1, 0), \quad \boldsymbol{r} = (0, 0, 1), \quad \boldsymbol{o} = (1, 0, 0).$$

Recap: R1CS

Example

Consider the most basic circuit with one multiplication gate: $x_1 \times x_2 = r$. The witness vector $\mathbf{z} = (r, x_1, x_2)$. So

$$z_2 \times z_3 = z_1$$

$$(0 + z_2 + 0) \times (0 + 0 + z_3) = z_1 + 0 + 0$$

$$(0z_1 + 1z_2 + 0z_3) \times (0z_1 + 0z_2 + 1z_3) = 1z_1 + 0z_2 + 0z_3$$

Therefore the coefficients vectors are:

$$\boldsymbol{\ell} = (0, 1, 0), \quad \boldsymbol{r} = (0, 0, 1), \quad \boldsymbol{o} = (1, 0, 0).$$

The general form of our constraint is:

$$(\ell_1 z_1 + \ell_2 z_2 + \ell_3 z_3)(r_1 z_1 + r_2 z_2 + r_3 z_3) = o_1 z_1 + o_2 z_2 + o_3 z_3$$

Recap: R1CS

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Recap: R1CS

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Thus, the next constraints can be built:

$$x_1 \times x_1 = x_1 \quad \text{(binary check)} \tag{1}$$

$$x_2 \times x_3 = \text{mult} \tag{2}$$

$$x_1 \times \text{mult} = \text{selectMult} \tag{3}$$

$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \tag{4}$$

Recap: R1CS

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Thus, the next constraints can be built:

$$x_1 \times x_1 = x_1 \quad \text{(binary check)} \tag{1}$$

$$x_2 \times x_3 = \text{mult} \tag{2}$$

$$x_1 \times \text{mult} = \text{selectMult} \tag{3}$$

$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \tag{4}$$

The witness vector: $\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$.

Recap: R1CS

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Thus, the next constraints can be built:

$$x_1 \times x_1 = x_1 \quad \text{(binary check)} \tag{1}$$
$$x_2 \times x_3 = \text{mult} \tag{2}$$
$$x_1 \times \text{mult} = \text{selectMult} \tag{3}$$
$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \tag{4}$$

The witness vector: $\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$.

The coefficients vectors:

$$\boldsymbol{\ell}_1 = (0,0,1,0,0,0,0), \quad \boldsymbol{r}_1 = (0,0,1,0,0,0,0), \quad \boldsymbol{o}_1 = (0,0,1,0,0,0,0)$$
$$\boldsymbol{\ell}_2 = (0,0,0,1,0,0,0), \quad \boldsymbol{r}_2 = (0,0,0,0,1,0,0), \quad \boldsymbol{o}_2 = (0,0,0,0,0,1,0)$$
$$\boldsymbol{\ell}_3 = (0,0,1,0,0,0,0), \quad \boldsymbol{r}_3 = (0,0,0,0,0,1,0), \quad \boldsymbol{o}_3 = (0,0,0,0,0,0,1)$$
$$\boldsymbol{\ell}_4 = (1,0,-1,0,0,0,0), \quad \boldsymbol{r}_4 = (0,0,0,1,1,0,0), \quad \boldsymbol{o}_4 = (0,1,0,0,0,0,-1)$$

QAP

Why do we need QAP?

Problems we have for now:

Why do we need QAP?

Problems we have for now:

✓ Although Rank-1 Constraint Systems provide a powerful method for representing computations, they are not succinct.

Why do we need QAP?

Problems we have for now:

- ✓ Although Rank-1 Constraint Systems provide a powerful method for representing computations, they are not succinct.
- ✓ We need to transform our computations into a form that is more convenient for proving statements about them.

Why do we need QAP?

Problems we have for now:

- ✓ Although Rank-1 Constraint Systems provide a powerful method for representing computations, they are not succinct.
- ✓ We need to transform our computations into a form that is more convenient for proving statements about them.

Notice

A very convenient form for representing computations is **polynomials**!

Idea: Instead of checking polynomial equality $P(T) = Q(T)$ at multiple points $x_1, \ldots, x_n$ (essentially, checking each constraint), we check it only once at $\tau \leftarrow\$ \mathbb{F}$: $P(\tau) = Q(\tau)$. Soundness is guaranteed by the **Schwartz-Zippel Lemma**.

R1CS in Matrix Form

We finished with the following constraint vectors:

$$\boldsymbol{\ell}_1, \boldsymbol{\ell}_2, \ldots, \boldsymbol{\ell}_m, \quad \boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_m, \quad \boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_m.$$

R1CS in Matrix Form

We finished with the following constraint vectors:

$$\boldsymbol{\ell}_1, \boldsymbol{\ell}_2, \ldots, \boldsymbol{\ell}_m, \quad \boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_m, \quad \boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_m.$$

Of course, they form corresponding matrices:

$$L = \begin{bmatrix} L_{1,1} & L_{1,2} & \ldots & L_{1,n} \\ L_{2,1} & L_{2,2} & \ldots & L_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ L_{m,1} & L_{m,2} & \ldots & L_{m,n} \end{bmatrix}, \text{ same goes for } R \text{ and } O$$

R1CS in Matrix Form

We finished with the following constraint vectors:

$$\boldsymbol{\ell}_1, \boldsymbol{\ell}_2, \ldots, \boldsymbol{\ell}_m, \quad \boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_m, \quad \boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_m.$$

Of course, they form corresponding matrices:

$$L = \begin{bmatrix} L_{1,1} & L_{1,2} & \ldots & L_{1,n} \\ L_{2,1} & L_{2,2} & \ldots & L_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ L_{m,1} & L_{m,2} & \ldots & L_{m,n} \end{bmatrix}, \text{ same goes for } R \text{ and } O$$

An example of a single “if” statement:

$$\begin{aligned} \boldsymbol{\ell}_1 &= (0,0,1,0,0,0,0) \\ \boldsymbol{\ell}_2 &= (0,0,0,1,0,0,0) \\ \boldsymbol{\ell}_3 &= (0,0,1,0,0,0,0) \\ \boldsymbol{\ell}_4 &= (1,0,-1,0,0,0,0) \end{aligned}$$

$$L = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

R1CS in Matrix Form

We finished with the following constraint vectors:

$$\boldsymbol{\ell}_1, \boldsymbol{\ell}_2, \ldots, \boldsymbol{\ell}_m, \quad \boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_m, \quad \boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_m.$$

Of course, they form corresponding matrices:

$$L = \begin{bmatrix} L_{1,1} & L_{1,2} & \ldots & L_{1,n} \\ L_{2,1} & L_{2,2} & \ldots & L_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ L_{m,1} & L_{m,2} & \ldots & L_{m,n} \end{bmatrix}, \text{ same goes for } R \text{ and } O$$

An example of a single “if” statement:

$$\begin{aligned} \boldsymbol{\ell}_1 &= (0,0,1,0,0,0,0) \\ \boldsymbol{\ell}_2 &= (0,0,0,1,0,0,0) \\ \boldsymbol{\ell}_3 &= (0,0,1,0,0,0,0) \\ \boldsymbol{\ell}_4 &= (1,0,-1,0,0,0,0) \end{aligned} \qquad L = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Pleeeeeenty of zeroes, right? And this is just one out of 3 matrices...

Columns of the Matrix

The previous witness vector:

$$z = (1, r, \overset{3}{x_1}, x_2, x_3, \text{mult}, \text{selectMult})$$

Let us take a closer look at the matrix columns:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Columns of the Matrix

The previous witness vector:

$$z = (1, r, \overset{3}{x_1}, x_2, x_3, \text{mult}, \text{selectMult})$$

Let us take a closer look at the matrix columns:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Consider 4th constraint: $(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult}$

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & \boxed{-1} & 0 & 0 & 0 & 0 \end{bmatrix}$$

Columns of the Matrix

The previous witness vector:

$$z = (1, r, \overset{3}{x_1}, x_2, x_3, \text{mult}, \text{selectMult})$$

Let us take a closer look at the matrix columns:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Consider 4th constraint: $(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult}$

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & \boxed{-1} & 0 & 0 & 0 & 0 \end{bmatrix}$$

So, every column is a mapping of constraint number to a coefficient for the witness element.

Polynomial Interpolation

As we know, such a mapping can be built using Lagrange interpolation polynomial with the following formula:

$$f(T) = \sum_{i=0}^{n} y_i e_i(T), \quad e_i(T) = \prod_{j=0, j\neq i}^{n} \frac{T - x_j}{x_i - x_j}.$$

Polynomial Interpolation

As we know, such a mapping can be built using Lagrange interpolation polynomial with the following formula:

$$f(T) = \sum_{i=0}^{n} y_i e_i(T), \quad e_i(T) = \prod_{j=0, j\neq i}^{n} \frac{T - x_j}{x_i - x_j}.$$

There are n columns and m constraints. So, it results in n polynomials such that:

$$\ell_j(i) = L_{i,j}, \ i \in \{1, \ldots, m\}, \ j \in \{1, \ldots, n\}$$

Polynomial Interpolation

As we know, such a mapping can be built using Lagrange interpolation polynomial with the following formula:

$$f(T) = \sum_{i=0}^{n} y_i e_i(T), \quad e_i(T) = \prod_{j=0, j\neq i}^{n} \frac{T - x_j}{x_i - x_j}.$$

There are n columns and m constraints. So, it results in n polynomials such that:

$$\ell_j(i) = L_{i,j}, \ i \in \{1, \ldots, m\}, \ j \in \{1, \ldots, n\}$$

The same is true for matrices R and O, with $3n$ polynomials in total, n for each of the coefficients matrices:

$$\ell_1(T), \ldots, \ell_n(T), r_1(T), \ldots, r_n(T), o_1(T), \ldots, o_n(T)$$

Polynomial Interpolation

As we know, such a mapping can be built using Lagrange interpolation polynomial with the following formula:

$$f(T) = \sum_{i=0}^{n} y_i e_i(T), \quad e_i(T) = \prod_{j=0, j\neq i}^{n} \frac{T - x_j}{x_i - x_j}.$$

There are n columns and m constraints. So, it results in n polynomials such that:

$$\ell_j(i) = L_{i,j}, \ i \in \{1, \ldots, m\}, \ j \in \{1, \ldots, n\}$$

The same is true for matrices R and O, with $3n$ polynomials in total, n for each of the coefficients matrices:

$$\ell_1(T), \ldots, \ell_n(T), r_1(T), \ldots, r_n(T), o_1(T), \ldots, o_n(T)$$

Note

We could have assigned any *unique* index from $\mathbb{F}$ to each constraint (say, t_i for each $i \in \{1, \ldots, m\}$) and interpolate through these points:

$$\ell_j(t_i) = L_{i,j}, \ i \in \{1, \ldots, m\}, \ j \in \{1, \ldots, n\}$$

Polynomial Interpolation: Example

Example

Considering the witness vector $\mathbf{z}$ and matrix L from the previous example, for the variable x_1, the next set of points can be derived:

$$\{(1,1),(2,0),(3,1),(4,-1)\}$$

Polynomial Interpolation: Example

Example

Considering the witness vector $\mathbf{z}$ and matrix L from the previous example, for the variable x_1, the next set of points can be derived:

$$\{(1,1),(2,0),(3,1),(4,-1)\}$$

The Lagrange interpolation polynomial for this set of points:

$$e_1(T) = -\frac{(T-2)(T-3)(T-4)}{6}, \quad e_2(T) = \frac{(T-1)(T-3)(T-4)}{2},$$
$$e_3(T) = -\frac{(T-1)(T-2)(T-4)}{2}, \quad e_4(T) = \frac{(T-1)(T-2)(T-3)}{6}.$$

Polynomial Interpolation: Example

Example

Considering the witness vector $\mathbf{z}$ and matrix L from the previous example, for the variable x_1, the next set of points can be derived:

$$\{(1,1),(2,0),(3,1),(4,-1)\}$$

The Lagrange interpolation polynomial for this set of points:

$$e_1(T) = -\frac{(T-2)(T-3)(T-4)}{6}, \quad e_2(T) = \frac{(T-1)(T-3)(T-4)}{2},$$
$$e_3(T) = -\frac{(T-1)(T-2)(T-4)}{2}, \quad e_4(T) = \frac{(T-1)(T-2)(T-3)}{6}.$$

Thus, the polynomial is given by:

$$\begin{aligned}\ell_{x_1}(T) &= 1 \cdot e_1(T) + 0 \cdot e_2(T) + 1 \cdot e_3(T) + (-1) \cdot e_4(T) \\ &= -\frac{5}{6}T^3 + 6T^2 - \frac{79}{6}T + 9\end{aligned}$$

Polynomial Interpolation: Example

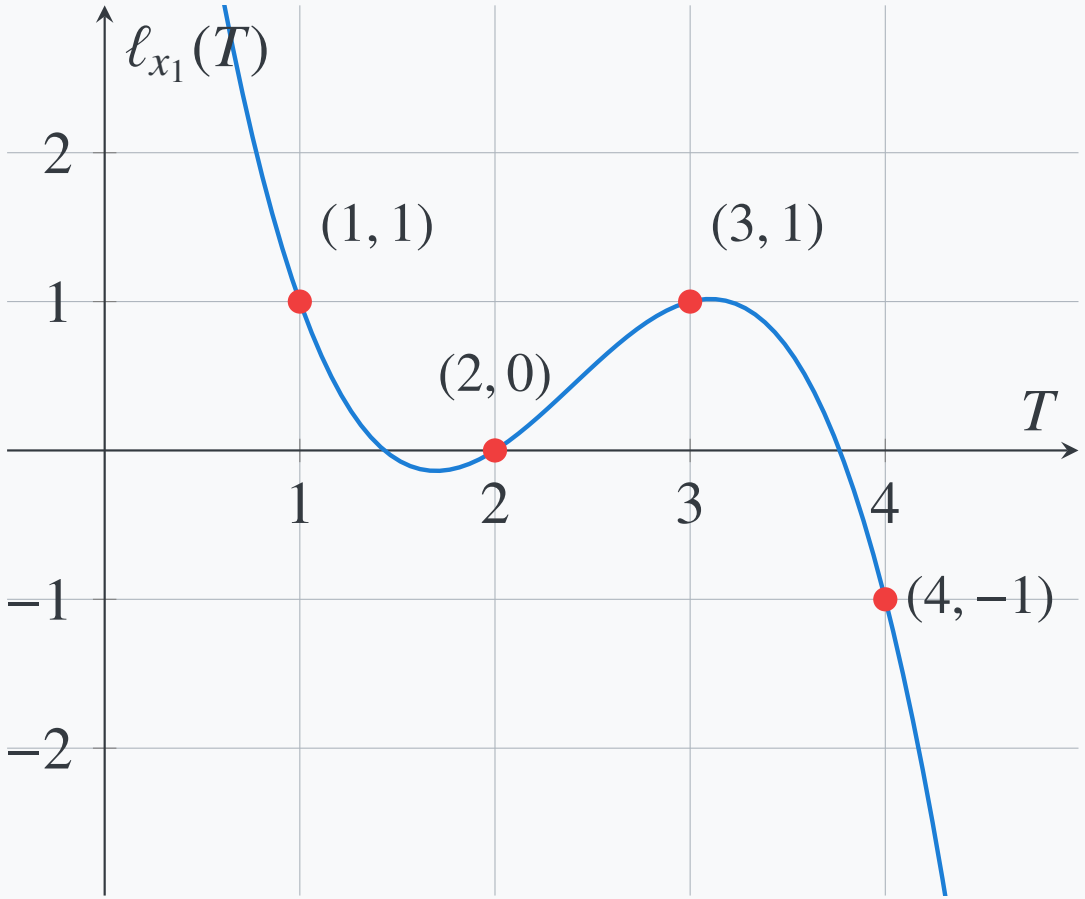


Illustration: The Lagrange interpolation polynomial for points $\{(1,1),(2,0),(3,1),(4,-1)\}$ visualized over $\mathbb{R}$.

Why Polynomials?

Question

But what does it change? We “exchanged” $3n$ columns for $3n$ polynomials.

Why Polynomials?

Question

But what does it change? We "exchanged" $3n$ columns for $3n$ polynomials.

Consider two polynomials $p(T)$ and $q(T)$:

$$p(T) = -\frac{1}{2}T^2 + \frac{3}{2}T, \qquad q(T) = \frac{1}{3}T^3 - 2T^2 + \frac{8}{3}T + 1.$$

With corresponding sets of points:

$$\{(0,0),(1,1),(2,1),(3,0)\}, \quad \{(0,1),(1,2),(2,1),(3,0)\}$$

Why Polynomials?

Question

But what does it change? We "exchanged" $3n$ columns for $3n$ polynomials.

Consider two polynomials $p(T)$ and $q(T)$:

$$p(T) = -\frac{1}{2}T^2 + \frac{3}{2}T, \qquad q(T) = \frac{1}{3}T^3 - 2T^2 + \frac{8}{3}T + 1.$$

With corresponding sets of points:

$$\{(0,0),(1,1),(2,1),(3,0)\}, \quad \{(0,1),(1,2),(2,1),(3,0)\}$$

The sum of these polynomials can be calculated as:

$$(p+q)(T) = \frac{1}{3}T^3 - \frac{5}{2}T^2 + \frac{25}{6}T + 1$$

The resulting polynomial $p + q$ corresponds to the set of points:

$$\{(0,1),(1,3),(2,2),(3,0)\}$$

Why Polynomials?

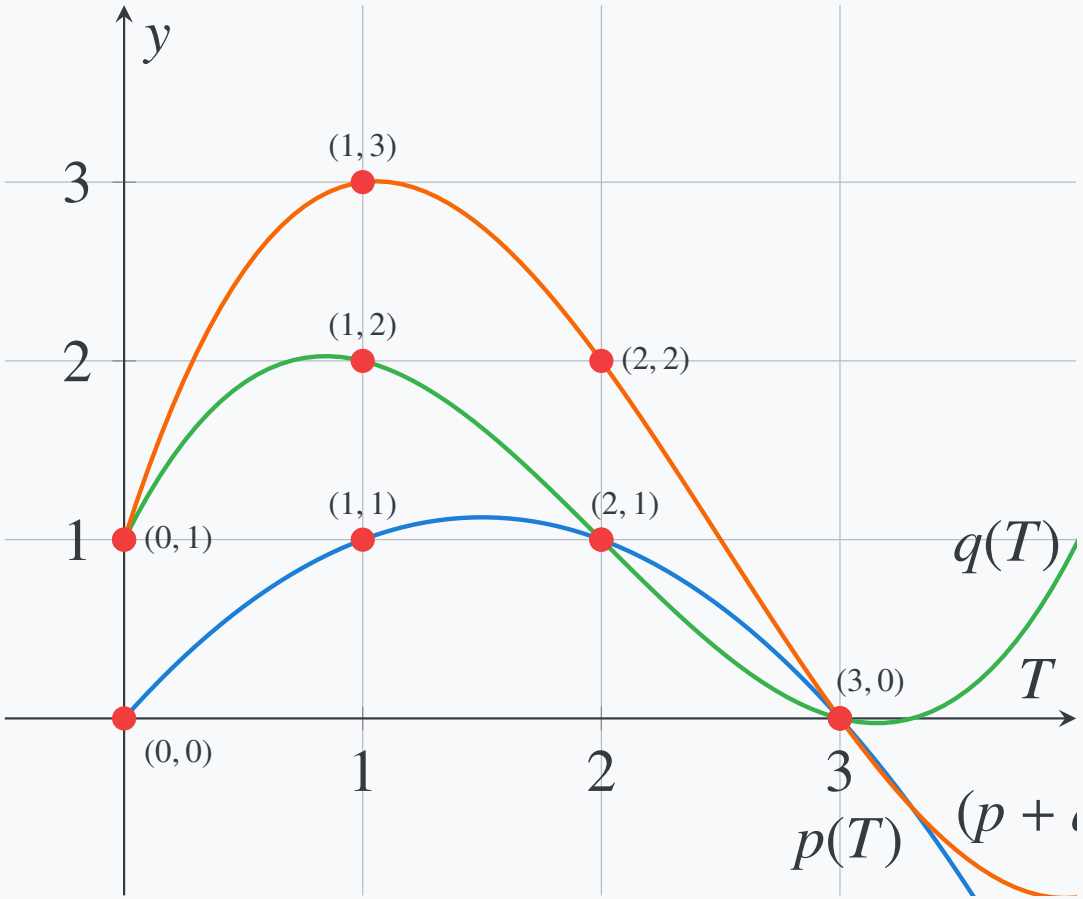


Figure: Addition of two polynomials

Putting It All Together

Now, using coefficients encoded with polynomials, we can build a constraint number $j \in \{1, \ldots, m\}$ in the next way:

$$\begin{aligned}(z_1\ell_1(j) + z_2\ell_2(j) + \cdots + z_n\ell_n(j))\times \\ \times(z_1 r_1(j) + z_2 r_2(j) + \cdots + z_n r_n(j)) = \\ =(z_1 o_1(j) + z_2 o_2(j) + \cdots + z_n o_n(j))\end{aligned}$$

Putting It All Together

Now, using coefficients encoded with polynomials, we can build a constraint number $j \in \{1, \ldots, m\}$ in the next way:

$$\begin{aligned}(z_1\ell_1(j) + z_2\ell_2(j) + \cdots + z_n\ell_n(j))\times \\ \times(z_1 r_1(j) + z_2 r_2(j) + \cdots + z_n r_n(j)) = \\ =(z_1 o_1(j) + z_2 o_2(j) + \cdots + z_n o_n(j))\end{aligned}$$

Or written more concisely:

$$\left(\sum_{i=1}^{n} z_i \ell_i(T)\right) \times \left(\sum_{i=1}^{n} z_i r_i(T)\right) = \left(\sum_{i=1}^{n} z_i o_i(T)\right)$$

Putting It All Together

Hold on, but why does it hold? Let us substitute any $T = j$ into this equation:

$$\left(\sum_{i=1}^{n} z_i \ell_i(j)\right) \times \left(\sum_{i=1}^{n} z_i r_i(j)\right) = \left(\sum_{i=1}^{n} z_i o_i(j)\right) \quad \textit{// for all } j \in \{1, \ldots, m\}$$

Putting It All Together

Hold on, but why does it hold? Let us substitute any $T = j$ into this equation:

$$\left(\sum_{i=1}^{n} z_i \ell_i(j)\right) \times \left(\sum_{i=1}^{n} z_i r_i(j)\right) = \left(\sum_{i=1}^{n} z_i o_i(j)\right) \quad \text{// for all } j \in \{1, \ldots, m\}$$

Recall that we interpolated polynomials to have $\ell_i(j) = L_{j,i}$.
Therefore, the equation above can be reduced to:

$$\left(\sum_{i=1}^{n} z_i L_{j,i}\right) \times \left(\sum_{i=1}^{n} z_i R_{j,i}\right) = \left(\sum_{i=1}^{n} z_i O_{j,i}\right) \quad \text{// for all } j \in \{1, \ldots, m\}$$

Putting It All Together

Hold on, but why does it hold? Let us substitute any $T = j$ into this equation:

$$\left(\sum_{i=1}^{n} z_i \ell_i(j)\right) \times \left(\sum_{i=1}^{n} z_i r_i(j)\right) = \left(\sum_{i=1}^{n} z_i o_i(j)\right) \quad \text{// for all } j \in \{1, \ldots, m\}$$

Recall that we interpolated polynomials to have $\ell_i(j) = L_{j,i}$.
Therefore, the equation above can be reduced to:

$$\left(\sum_{i=1}^{n} z_i L_{j,i}\right) \times \left(\sum_{i=1}^{n} z_i R_{j,i}\right) = \left(\sum_{i=1}^{n} z_i O_{j,i}\right) \quad \text{// for all } j \in \{1, \ldots, m\}$$

But hold on again! Notice that $\sum_{i=1}^{n} z_i L_{j,i} = \langle \mathbf{z}, \boldsymbol{\ell}_j \rangle$ and therefore we have:

$$\langle \mathbf{z}, \boldsymbol{\ell}_j \rangle \times \langle \mathbf{z}, \boldsymbol{r}_j \rangle = \langle \mathbf{z}, \boldsymbol{o}_j \rangle \quad \text{// for all } j \in \{1, \ldots, m\},$$

so we ended up with the initial m constraint equations!

Vanishing Polynomial

Now let us define polynomials $\ell(T)$, $r(T)$, $o(T)$ for easier notation:

$$\ell(T) = \sum_{i=1}^{n} z_i \ell_i(T), \quad r(T) = \sum_{i=1}^{n} z_i r_i(T), \quad o(T) = \sum_{i=1}^{n} z_i o_i(T)$$

Vanishing Polynomial

Now let us define polynomials $\ell(T)$, $r(T)$, $o(T)$ for easier notation:

$$\ell(T) = \sum_{i=1}^{n} z_i \ell_i(T), \quad r(T) = \sum_{i=1}^{n} z_i r_i(T), \quad o(T) = \sum_{i=1}^{n} z_i o_i(T)$$

Therefore:

$$\ell(j) \times r(j) = o(j) \quad \textit{// for all } j \in \{1, \ldots, m\}$$

Now, we can define a polynomial $m(T)$, that has zeros at all elements from the set $\Omega = \{1, \ldots, m\}$

$$m(T) = \ell(T) \times r(T) - o(T)$$

Vanishing Polynomial

Now let us define polynomials $\ell(T)$, $r(T)$, $o(T)$ for easier notation:

$$\ell(T) = \sum_{i=1}^{n} z_i \ell_i(T), \quad r(T) = \sum_{i=1}^{n} z_i r_i(T), \quad o(T) = \sum_{i=1}^{n} z_i o_i(T)$$

Therefore:

$$\ell(j) \times r(j) = o(j) \quad \textit{// for all } j \in \{1, \ldots, m\}$$

Now, we can define a polynomial $m(T)$, that has zeros at all elements from the set $\Omega = \{1, \ldots, m\}$

$$m(T) = \ell(T) \times r(T) - o(T)$$

It means, that $m(T)$ can be divided by **vanishing polynomial** $z_\Omega(T)$ without a remainder!

$$z_\Omega(T) = \prod_{i=1}^{m} (T - i), \qquad h(T) = \frac{m(T)}{z_\Omega(T)} \text{ is a polynomial}$$

Quadratic Arithmetic Program

Definition (Quadratic Arithmetic Program)

Suppose that m R1CS constraints with a witness of size n are written in a form $L\mathbf{z} \odot R\mathbf{z} = O\mathbf{z}$ for $L, R, O \in \mathbb{F}^{m \times n}$.
Then, the **Quadratic Arithmetic Program** consists of $3n$ polynomials $\ell_1, \ldots, \ell_n, r_1, \ldots, r_n, o_1, \ldots, o_n$ such that:

$$\ell_j(i) = L_{i,j},\ r_j(i) = R_{i,j},\ o_j(i) = O_{i,j} \quad \text{// for all } i \in \{1, \ldots, m\}, j \in \{1, \ldots, n\}$$

Then, $\mathbf{z} \in \mathbb{F}^n$ is a valid assignment for the given QAP and **target polynomial** $z(T) = \prod_{i=1}^{m}(T - i)$ if and only if there exists such a polynomial $h(T)$ such that

$$\left(\sum_{i=1}^{n} z_i \ell_i(T)\right)\left(\sum_{i=1}^{n} z_i r_i(T)\right) - \left(\sum_{i=1}^{n} z_i o_i(T)\right) = z(T)h(T)$$

Probabilistically Checkable Proofs

Probabilistically Checkable Proofs

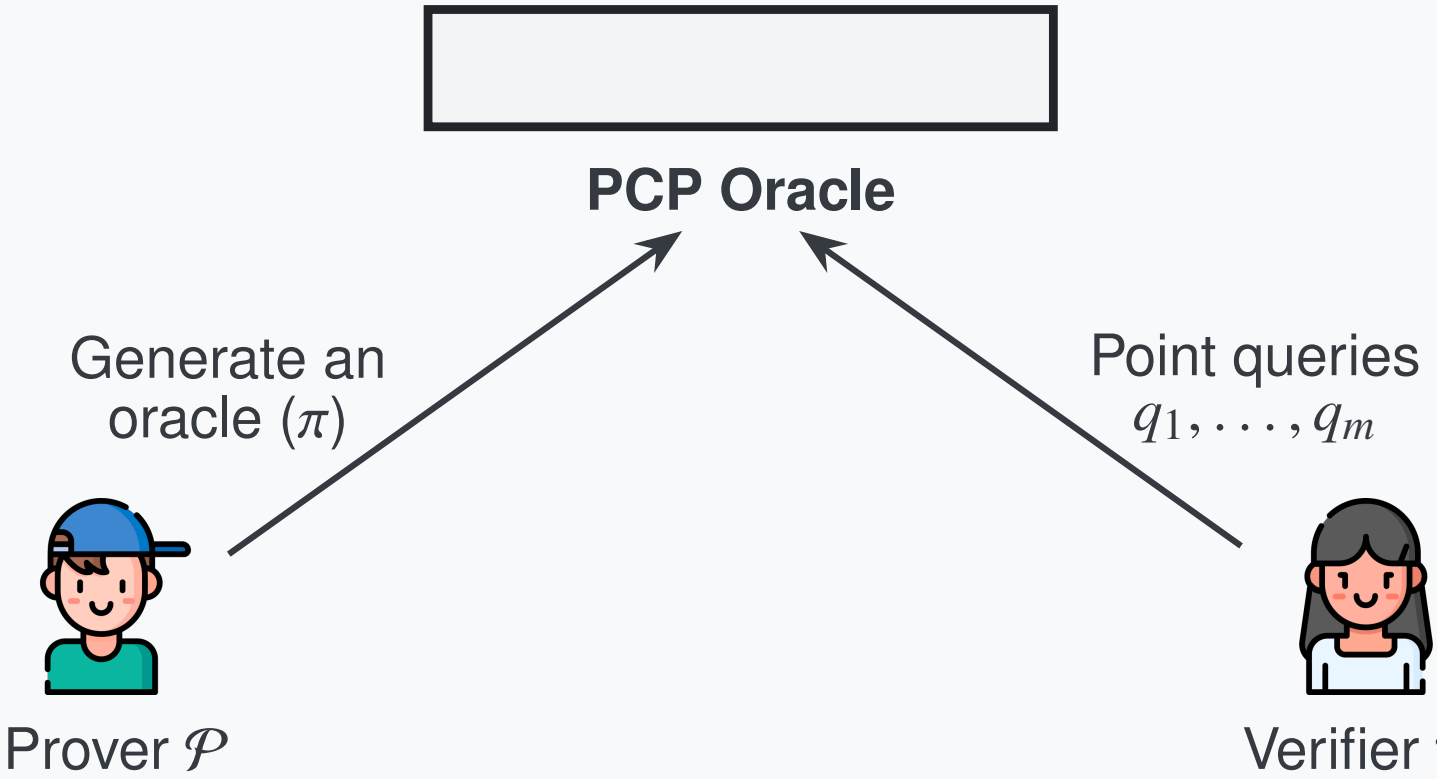


Figure: Illustration of a Probabilistically Checkable Proof (PCP) system. The prover $\mathcal{P}$ generates a PCP oracle π that is queried by the verifier $\mathcal{V}$ at specific points $q_1, \ldots, q_m$.

Probabilistically Checkable Proofs

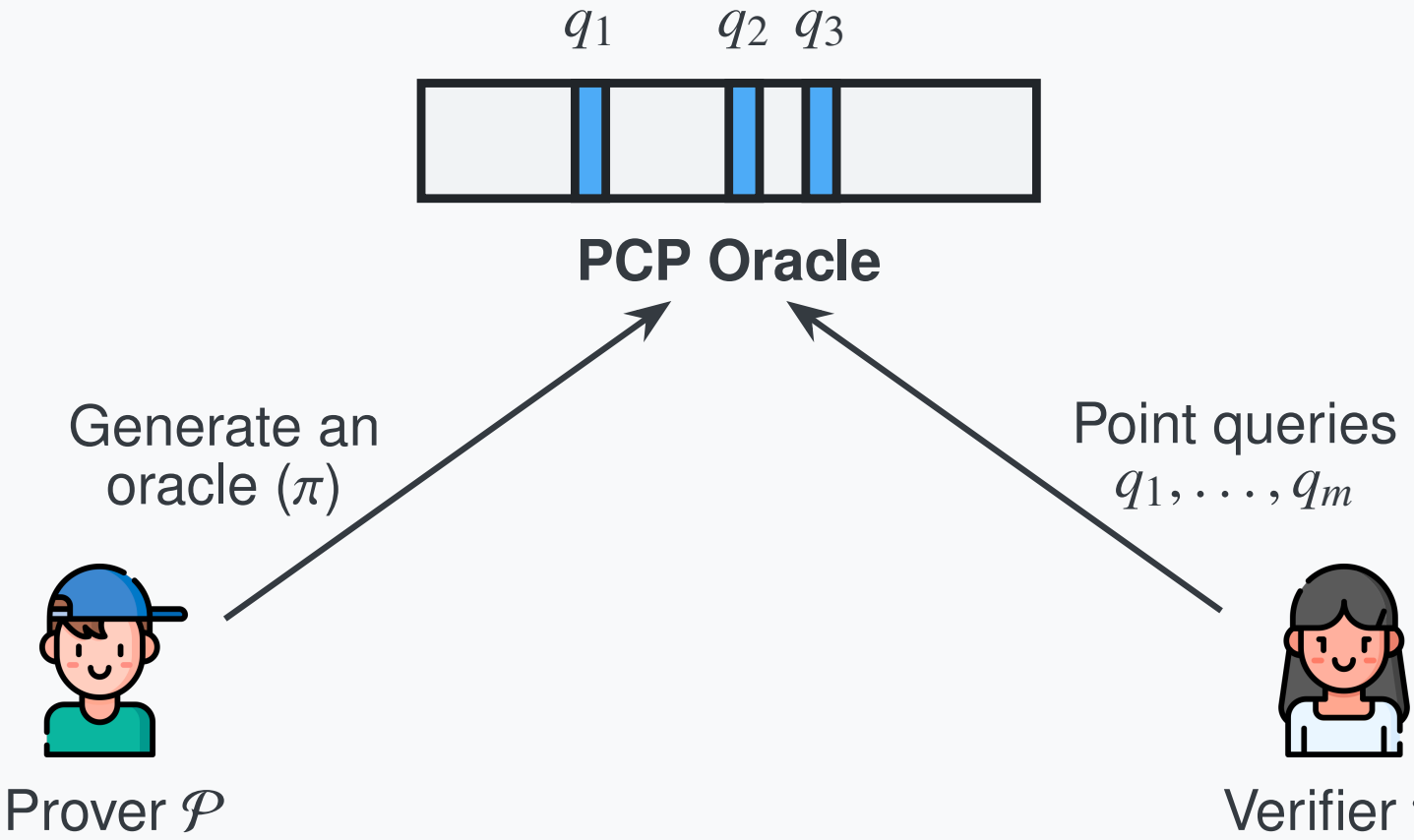


Figure: Illustration of a Probabilistically Checkable Proof (PCP) system. The prover $\mathcal{P}$ generates a PCP oracle π that is queried by the verifier $\mathcal{V}$ at specific points $q_1, \ldots, q_m$.

Extensions of PCPs

Three main extensions of PCPs that are frequently used in SNARKs are:

- **IPCP** (**Interactive PCP**): The prover commits to the PCP oracle and then, based on the interaction between the prover and verifier, the verifier queries the oracle and decides whether to accept the proof.

Extensions of PCPs

Three main extensions of PCPs that are frequently used in SNARKs are:

- **IPCP** (**Interactive PCP**): The prover commits to the PCP oracle and then, based on the interaction between the prover and verifier, the verifier queries the oracle and decides whether to accept the proof.
- **IOP** (**Interactive Oracle Proof**): The prover and verifier interact and on each round, the prover commits to a new oracle. The verifier queries the oracle and decides whether to accept the proof.

Extensions of PCPs

Three main extensions of PCPs that are frequently used in SNARKs are:

- **IPCP** (**Interactive PCP**): The prover commits to the PCP oracle and then, based on the interaction between the prover and verifier, the verifier queries the oracle and decides whether to accept the proof.
- **IOP** (**Interactive Oracle Proof**): The prover and verifier interact and on each round, the prover commits to a new oracle. The verifier queries the oracle and decides whether to accept the proof.
- **LPCP** (**Linear PCP**): The prover commits to a linear function and the verifier queries the function at specific points.

Interactive Oracle Proofs

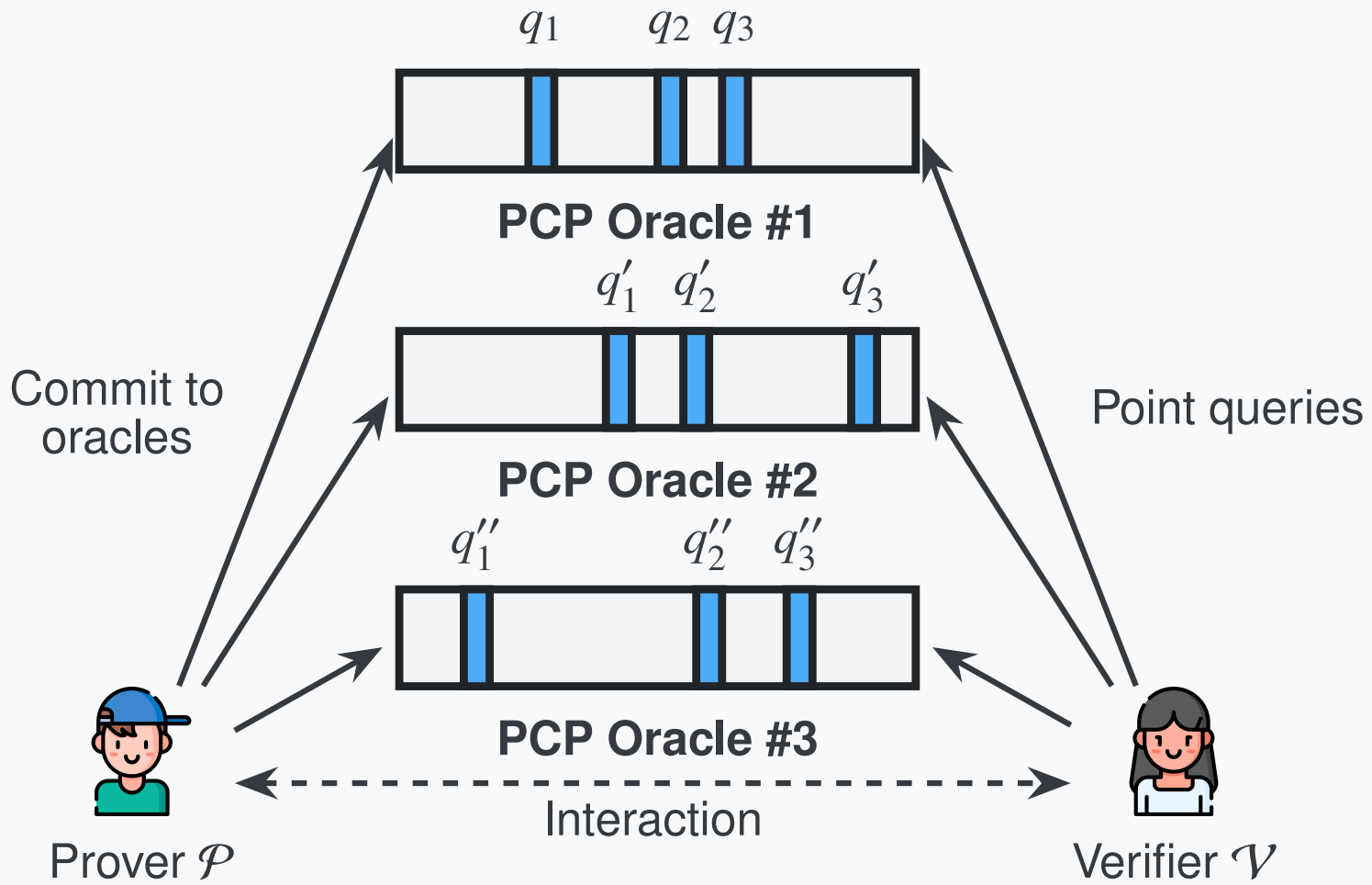


Figure: Illustration of an Interactive Oracle Proof (IOP). On each round i $(1 \leq i \leq r)$, $\mathcal{V}$ sends a message m_i, and $\mathcal{P}$ commits to a new oracle π_i, which $\mathcal{V}$ can query at $\mathbf{q}_i = (q_{i,1}, \ldots, q_{i,m})$.

QAP as a Linear PCP

Linear PCP

Definition (Linear PCP)

A **Linear PCP** is a PCP where the prover commits to a linear function $\pi = (\pi_1, \ldots, \pi_k)$ and the verifier queries the function at specific points $\mathbf{q}_1, \ldots, \mathbf{q}_r$. Then, the prover responds with the values of the function at these points:

$$\langle \pi_1, \mathbf{q}_1 \rangle, \langle \pi_2, \mathbf{q}_2 \rangle, \ldots, \langle \pi_r, \mathbf{q}_r \rangle.$$

QAP as a Linear PCP

Example (QAP as a Linear PCP)

Recall that key QAP equation is:

$$\ell(T) \times r(T) - o(T) = z(T)h(T).$$

Denote $\boldsymbol{\ell}(T) = (\ell_1(T), \ldots, \ell_n(T))$, and similarly $\boldsymbol{r}(T)$, $\boldsymbol{o}(T)$.

QAP as a Linear PCP

Example (QAP as a Linear PCP)

Recall that key QAP equation is:

$$\ell(T) \times r(T) - o(T) = z(T)h(T).$$

Denote $\boldsymbol{\ell}(T) = (\ell_1(T), \ldots, \ell_n(T))$, and similarly $\boldsymbol{r}(T)$, $\boldsymbol{o}(T)$.
Now, consider the following **linear PCP for QAP**:

1: $\mathcal{P}$ commits to an extended witness $\mathbf{z}$ and coefficients $\mathbf{h} = (h_1, \ldots, h_n)$ of $h(T)$.

QAP as a Linear PCP

Example (QAP as a Linear PCP)

Recall that key QAP equation is:

$$\ell(T) \times r(T) - o(T) = z(T)h(T).$$

Denote $\boldsymbol{\ell}(T) = (\ell_1(T), \ldots, \ell_n(T))$, and similarly $\boldsymbol{r}(T)$, $\boldsymbol{o}(T)$.
Now, consider the following **linear PCP for QAP**:

1: $\mathcal{P}$ commits to an extended witness $\mathbf{z}$ and coefficients $\mathbf{h} = (h_1, \ldots, h_n)$ of $h(T)$.
2: $\mathcal{V}$ samples $\gamma \leftarrow\$ \mathbb{F}$ and sends query $\boldsymbol{\gamma} = (\gamma, \gamma^2, \ldots, \gamma^n)$ to $\mathcal{P}$.

QAP as a Linear PCP

Example (QAP as a Linear PCP)

Recall that key QAP equation is:

$$\ell(T) \times r(T) - o(T) = z(T)h(T).$$

Denote $\boldsymbol{\ell}(T) = (\ell_1(T), \ldots, \ell_n(T))$, and similarly $\boldsymbol{r}(T)$, $\boldsymbol{o}(T)$.
Now, consider the following **linear PCP for QAP**:

1: $\mathcal{P}$ commits to an extended witness $\mathbf{z}$ and coefficients $\mathbf{h} = (h_1, \ldots, h_n)$ of $h(T)$.
2: $\mathcal{V}$ samples $\gamma \leftarrow\!\!\$\ \mathbb{F}$ and sends query $\boldsymbol{\gamma} = (\gamma, \gamma^2, \ldots, \gamma^n)$ to $\mathcal{P}$.
3: $\mathcal{P}$ reveals the following values:

$$\pi_1 \leftarrow \langle \mathbf{z}, \boldsymbol{\ell}(\gamma) \rangle, \qquad \pi_2 \leftarrow \langle \mathbf{z}, \boldsymbol{r}(\gamma) \rangle,$$
$$\pi_3 \leftarrow \langle \mathbf{z}, \boldsymbol{o}(\gamma) \rangle, \qquad \pi_4 \leftarrow z(\gamma) \cdot \langle \mathbf{h}, \boldsymbol{\gamma} \rangle.$$

QAP as a Linear PCP

Example (QAP as a Linear PCP)

Recall that key QAP equation is:

$$\ell(T) \times r(T) - o(T) = z(T)h(T).$$

Denote $\boldsymbol{\ell}(T) = (\ell_1(T), \ldots, \ell_n(T))$, and similarly $\boldsymbol{r}(T)$, $\boldsymbol{o}(T)$.
Now, consider the following **linear PCP for QAP**:

1: $\mathcal{P}$ commits to an extended witness $\mathbf{z}$ and coefficients $\mathbf{h} = (h_1, \ldots, h_n)$ of $h(T)$.
2: $\mathcal{V}$ samples $\gamma \leftarrow\!\!\$\ \mathbb{F}$ and sends query $\boldsymbol{\gamma} = (\gamma, \gamma^2, \ldots, \gamma^n)$ to $\mathcal{P}$.
3: $\mathcal{P}$ reveals the following values:

$$\pi_1 \leftarrow \langle \mathbf{z}, \boldsymbol{\ell}(\gamma) \rangle, \qquad \pi_2 \leftarrow \langle \mathbf{z}, \boldsymbol{r}(\gamma) \rangle,$$
$$\pi_3 \leftarrow \langle \mathbf{z}, \boldsymbol{o}(\gamma) \rangle, \qquad \pi_4 \leftarrow z(\gamma) \cdot \langle \mathbf{h}, \boldsymbol{\gamma} \rangle.$$

4: $\mathcal{V}$ checks whether $\pi_1\pi_2 - \pi_3 = \pi_4$.

Why is it Sound?

Question

Why is it safe to use such a check? (assuming proper commitments).

Why is it Sound?

Question

Why is it safe to use such a check? (assuming proper commitments).

The polynomials $\ell(T)$, $r(T)$ and $o(T)$ are interpolated polynomials using $|C|$ (number of gates) points, so:

$$\deg(\ell) \leq |C|, \quad \deg(r) \leq |C|, \quad \deg(o) \leq |C|$$

Why is it Sound?

Question

Why is it safe to use such a check? (assuming proper commitments).

The polynomials $\ell(T)$, $r(T)$ and $o(T)$ are interpolated polynomials using $|C|$ (number of gates) points, so:

$$\deg(\ell) \leq |C|, \quad \deg(r) \leq |C|, \quad \deg(o) \leq |C|$$

Thus, we can estimate the degree of polynomial $m(T) = \ell(T)r(T) - o(T)$.

$$\deg(m) \leq \max\{\deg(\ell) + \deg(r), \deg(o)\} \leq 2|C|$$

Why is it Sound?

Question

Why is it safe to use such a check? (assuming proper commitments).

The polynomials $\ell(T)$, $r(T)$ and $o(T)$ are interpolated polynomials using $|C|$ (number of gates) points, so:

$$\deg(\ell) \leq |C|, \quad \deg(r) \leq |C|, \quad \deg(o) \leq |C|$$

Thus, we can estimate the degree of polynomial $m(T) = \ell(T)r(T) - o(T)$.

$$\deg(m) \leq \max\{\deg(\ell) + \deg(r), \deg(o)\} \leq 2|C|$$

If an adversary $\mathcal{A}$ does not know a valid witness $\mathbf{z}$, he can compute a polynomial $(\widetilde{m}(T), \widetilde{h}(T)) \leftarrow \mathcal{A}(\cdot)$ that satisfies a verifier $\mathcal{V}$ with probability:

$$\Pr_{s \leftarrow \$\mathbb{F}}[\widetilde{m}(s) = z(s)\widetilde{h}(s)] \leq \frac{2|C|}{\#\mathbb{F}}$$

If $\#\mathbb{F}$ is large enough, $2|C|/\#\mathbb{F}$ is *negligible*.

Proof Of Exponent

Encrypted Verification

Let us try to prove that we know some polynomial $p(T)$ that can be divided by $t(T)$ without a remainder.

Encrypted Verification

Let us try to prove that we know some polynomial $p(T)$ that can be divided by $t(T)$ without a remainder.

Consider the polynomial: $p(T) = T^2 - 5T + 2$. Additionally, we will need a cyclic group $\mathbb{G}$ with a generator $g \in \mathbb{G}$. We also define the *encryption* operation as follows:

$$\text{Enc} : \mathbb{F} \to \mathbb{G}, \quad \text{Enc}(x) := g^x$$

Encrypted Verification

Let us try to prove that we know some polynomial $p(T)$ that can be divided by $t(T)$ without a remainder.

Consider the polynomial: $p(T) = T^2 - 5T + 2$. Additionally, we will need a cyclic group $\mathbb{G}$ with a generator $g \in \mathbb{G}$. We also define the *encryption* operation as follows:

$$\text{Enc} : \mathbb{F} \to \mathbb{G}, \quad \text{Enc}(x) := g^x$$

Essentially, $\text{Enc}(p(\tau))$ is the **KZG Commitment**. Let us see the encryption of $p(\tau)$ for our example:

$$\text{Enc}(p(\tau)) = g^{p(\tau)} = g^{(\tau^2 - 5\tau + 2)} = \left(g^{\tau^2}\right)^1 \cdot \left(g^{\tau^1}\right)^{-5} \cdot \left(g^{\tau^0}\right)^2$$

Encrypted Verification

Let us try to prove that we know some polynomial $p(T)$ that can be divided by $t(T)$ without a remainder.

Consider the polynomial: $p(T) = T^2 - 5T + 2$. Additionally, we will need a cyclic group $\mathbb{G}$ with a generator $g \in \mathbb{G}$. We also define the *encryption* operation as follows:

$$\text{Enc} : \mathbb{F} \to \mathbb{G}, \quad \text{Enc}(x) := g^x$$

Essentially, $\text{Enc}(p(\tau))$ is the **KZG Commitment**. Let us see the encryption of $p(\tau)$ for our example:

$$\text{Enc}(p(\tau)) = g^{p(\tau)} = g^{(\tau^2 - 5\tau + 2)} = \left(g^{\tau^2}\right)^1 \cdot \left(g^{\tau^1}\right)^{-5} \cdot \left(g^{\tau^0}\right)^2$$

Note

KZG Commitment requires only encrypted powers of τ: $\{g^{\tau^i}\}_{i \in [d]}$.

Encrypted Verification

Verifier:

✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.

Encrypted Verification

Verifier:

- ✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Encrypted Verification

Verifier:

- ✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.

Encrypted Verification

Verifier:

- ✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.
- ✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Encrypted Verification

Verifier:

✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

Encrypted Verification

Verifier:

✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$ and $g^{h(\tau)}$.

Encrypted Verification

Verifier:

✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$ and $g^{h(\tau)}$.

✓ Provides encrypted polynomials $g^{p(\tau)}$ and $g^{h(\tau)}$ to the verifier.

Encrypted Verification

Verifier:

✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$ and $g^{h(\tau)}$.

✓ Provides encrypted polynomials $g^{p(\tau)}$ and $g^{h(\tau)}$ to the verifier.

Verifier:

✓ Checks whether $g^{p(\tau)} = \left(g^{h(\tau)}\right)^{t(\tau)}$.

That doesn't work...

Verifier:

✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.

That doesn't work...

Verifier:

✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

That doesn't work...

Verifier:

- ✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.

That doesn't work...

Verifier:

- ✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.
- ✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

That doesn't work...

Verifier:

✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Adversary:

✓ Picks a random value $r \leftarrow\$ \mathbb{F}$, calculates g^r.

That doesn't work...

Verifier:

✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Adversary:

✓ Picks a random value $r \leftarrow\$ \mathbb{F}$, calculates g^r.

✓ Calculates $g^{t(\tau)}$.

That doesn't work...

Verifier:

- ✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.
- ✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Adversary:

- ✓ Picks a random value $r \leftarrow\$ \mathbb{F}$, calculates g^r.
- ✓ Calculates $g^{t(\tau)}$.
- ✓ Calculates $g^{\widetilde{p}(\tau)} = \left(g^{t(\tau)}\right)^r$.

That doesn't work...

Verifier:

✓ Picks a random value $\tau \leftarrow\$ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

✓ Outputs prover parameters $\{g^{\tau^i}\}_{i\in[d]}$.

Adversary:

✓ Picks a random value $r \leftarrow\$ \mathbb{F}$, calculates g^r.

✓ Calculates $g^{t(\tau)}$.

✓ Calculates $g^{\widetilde{p}(\tau)} = \left(g^{t(\tau)}\right)^r$.

Verifier:

✓ Checks whether $g^{\widetilde{p}(\tau)} = (g^r)^{t(\tau)}$.

Proof Of Exponent

Verifier:

✓ Picks random values $\tau \leftarrow\!\!\$\ \mathbb{F}$, $\alpha \leftarrow\!\!\$\ \mathbb{F}$.

Proof Of Exponent

Verifier:

- ✓ Picks random values $\tau \leftarrow\!\$\ \mathbb{F}$, $\alpha \leftarrow\!\$\ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.

Proof Of Exponent

Verifier:

- ✓ Picks random values $\tau \leftarrow\$ \mathbb{F}$, $\alpha \leftarrow\$ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.

Proof Of Exponent

Verifier:

✓ Picks random values $\tau \leftarrow\!\$\ \mathbb{F}$, $\alpha \leftarrow\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

Proof Of Exponent

Verifier:

✓ Picks random values $\tau \leftarrow\!\!\$\ \mathbb{F}$, $\alpha \leftarrow\!\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$, $g^{h(\tau)}$.

Proof Of Exponent

Verifier:

✓ Picks random values $\tau \leftarrow\!\$\ \mathbb{F}$, $\alpha \leftarrow\!\$\ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$, $g^{h(\tau)}$.

✓ Using $\{g^{\alpha\tau^i}\}_{i\in[d]}$ calculates $g^{p'(\tau)} = g^{\alpha p(\tau)}$.

Proof Of Exponent

Verifier:

- ✓ Picks random values $\tau \leftarrow\$ \mathbb{F}$, $\alpha \leftarrow\$ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.
- ✓ Calculates $t(\tau)$.

Prover:

- ✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.
- ✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$, $g^{h(\tau)}$.
- ✓ Using $\{g^{\alpha\tau^i}\}_{i\in[d]}$ calculates $g^{p'(\tau)} = g^{\alpha p(\tau)}$.
- ✓ Provides encrypted polynomials to the verifier.

Proof Of Exponent

Verifier:

✓ Picks random values $\tau \leftarrow\$ \mathbb{F}$, $\alpha \leftarrow\$ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$, $g^{h(\tau)}$.

✓ Using $\{g^{\alpha\tau^i}\}_{i\in[d]}$ calculates $g^{p'(\tau)} = g^{\alpha p(\tau)}$.

✓ Provides encrypted polynomials to the verifier.

Verifier:

✓ Checks whether $g^{p(\tau)} = \left(g^{h(\tau)}\right)^{t(\tau)}$.

Proof Of Exponent

Verifier:

✓ Picks random values $\tau \leftarrow\$ \mathbb{F}$, $\alpha \leftarrow\$ \mathbb{F}$.

✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$.

✓ Calculates $t(\tau)$.

Prover:

✓ Calculates $h(T) = \frac{p(T)}{t(T)}$.

✓ Using $\{g^{\tau^i}\}_{i\in[d]}$ calculates $g^{p(\tau)}$, $g^{h(\tau)}$.

✓ Using $\{g^{\alpha\tau^i}\}_{i\in[d]}$ calculates $g^{p'(\tau)} = g^{\alpha p(\tau)}$.

✓ Provides encrypted polynomials to the verifier.

Verifier:

✓ Checks whether $g^{p(\tau)} = \left(g^{h(\tau)}\right)^{t(\tau)}$.

✓ Checks whether $g^{p'(\tau)} = \left(g^{p(\tau)}\right)^{\alpha} = g^{\alpha p(\tau)}$.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream